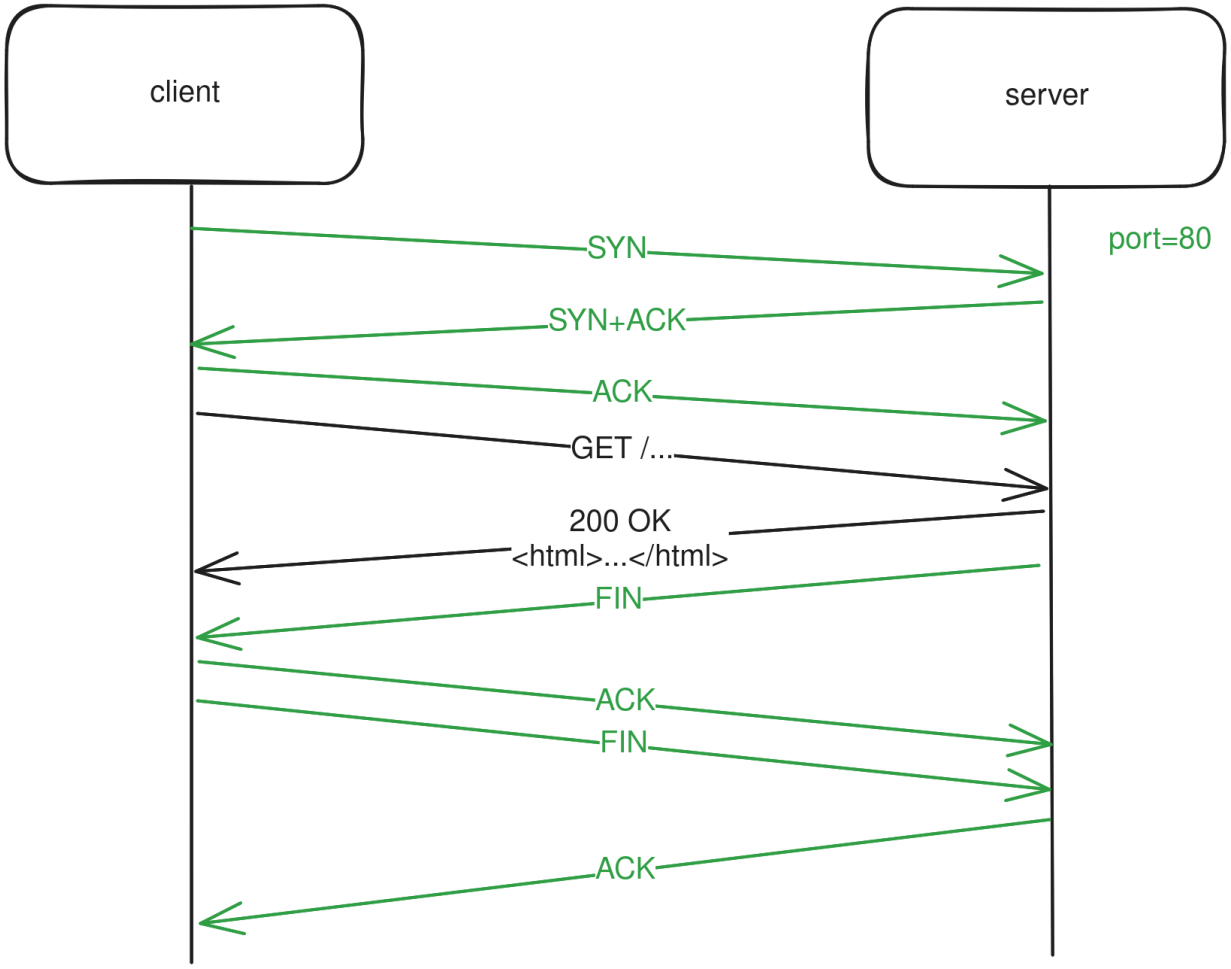
INFOB315 - Télécommunications et réseaux
Client HTTP en Python

1 Client HTTP

Le but de ce TP est de créer un mini client HTTP en Python. Ce client reproduira les étapes réalisées par un navigateur internet afin de récupérer le contenu d'une page HTML. La figure 1 reprend les échanges entre le client (votre code Python) et le serveur lors de la récupération d'une page HTML. Les échanges en lien avec le protocole TCP sont représentés au moyen de flèches vertes et les échanges utilisant le protocole HTTP avec des flèches noires.

Figure 1 – Échanges entre le client et le serveur



Trois grandes étapes sont ainsi identifiables. L'établissement de la connexion TCP avec le *Three-way handshake*, la requête et la réponse HTTP ainsi que la fermeture de la connexion TCP.

1.1 Les sockets

Les *sockets* sont un moyen pour le système d'exploitation de représenter une connexion ouverte par un processus. Pour vous connecter en HTTP à un serveur, il sera nécessaire de créer un *socket* représentant un la connexion. Une fois créé, un *socket* permet de se connecter à un serveur en spécifiant son IP (ou son nom de domaine) ainsi que le port sur lequel le serveur écoute. Dans le cas de HTTP, le port d'un serveur est habituellement le port 80. Une fois la connexion ouverte, le *socket* permet d'envoyer et de recevoir des données vers et depuis le serveur distant. Dans le cadre de ce TP, les données envoyées seront donc la requête HTTP et les données reçues seront la réponse HTTP associée. De la même façon qu'il est important de fermer un fichier ouvert dans un programme, il est important de fermer le *socket* une fois l'échange terminé.

Afin de manipuler des *sockets* en Python, le module `socket` est disponible. La documentation de ce dernier est disponible en ligne (https://docs.python.org/3/library/socket.html). Dans ce nodule, les fonctions et méthodes dont il serait utile de regarder la documentation afin de réaliser ce TP sont :

- `socket()`
- `socket.connect()`
- `socket.send()`
- `socket.recv()`
- `socket.close()`

1.2 Code Python

En Python, implémentez la fonction `get_http(url: str) -> (int, dict[str, str], str)` prenant en paramètre l'URL de la page HTML à récupérer et qui retourne le code de statut de la réponse, un dictionnaire des *headers* et le *body* (dans ce cas la page HTML). Le code suivant montre un exemple d'utilisation de cette fonction.

```python
url="gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file1.html"
status_code, headers, body = get_http(url)

print(f"Status Code: {status_code}")

print("Headers:")
for header, value in headers.items():
    print(f"\t{header} -> {value}")

print("Body:")
print("\t" + body.replace('\n', '\n\t'))
```

L'exécution de ce code, une fois la fonction `get_http` implémentée, devrait produire le résultat suivant :

```
Status Code: 200
Headers:
        Date -> Tue, 07 Nov 2023 14:24:01 GMT
        Server -> Apache/2.4.6 (CentOS) OpenSSL/1.0.2k-fips PHP/7.4.33 mod_perl/2.0.11 Perl/
    v5.16.3
        Last-Modified -> Tue, 07 Nov 2023 06:59:02 GMT
        ETag -> "80-6098a81722b66"
        Accept-Ranges -> bytes
        Content-Length -> 128
        Content-Type -> text/html; charset=UTF-8
Body:
        <html>
        Congratulations.  You've downloaded the file
        http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file1.html!
        </html>
```